Model Card — Movement Anomaly Detector (`ml/`)

> **One-line summary.** An **unsupervised** IsolationForest that assigns each GPS point a **movement anomaly score** (higher = more unusual relative to normal GeoLife movement). It is **not** a danger classifier and its score is **not a probability**. It supplements — it does not replace — the app's deterministic geo-fencing.
>
> **This card covers the trajectory model only.** The project has **two other** things that are sometimes called "the AI", and neither is described here:
>
> - **Activity recognition and impact/fall-signature detection** (two *supervised* random forests over 24 accelerometer/gyroscope features, trained on real UCI HAR data and shipped inside the APK as JSON walked by hand-written Kotlin) — see `model_card_movement.md`. That card also states plainly that UCI HAR contains **no falls**, so the impact detector's positives are **synthesised impacts injected into real recordings** and its metrics are **not** a clinical fall-detection claim.
> - **The approach-risk score** shown when the tourist nears a restricted area — see §9.3 of `README.md` or §7.8 of `final_project_report.md`. It is a **published weighted checklist, not a model**: there is no labelled outcome data to fit, and it is labelled as such in the UI.

---

1. Model details

Field	Value
Kind	`sklearn.ensemble.IsolationForest` (unsupervised) + `StandardScaler`
Task	Unsupervised anomaly **scoring** of movement features
Output	`movement_anomaly_score = -score_samples(scaled_features)`, **higher = more anomalous**
`n_estimators`	200
`contamination`	0.02 (used only to pick a flagging threshold, not to define truth)
`max_samples`	`auto`
Random seed	42 (reproducible bit-for-bit)
Artifact	`ml/models/isolation_forest.joblib` (model + scaler + feature list + meta)

Field	Value
Code	`ml/train.py` (fit), `ml/evaluate.py` (validation), `ml/predict.py` (inference)

Supervised? No. **Labels used for training?** None (`model_meta.json → "labels_used": null`).

---

2. Intended use

- Rank points/trajectories by *how unusual* their movement pattern is, for human review and as an **additional, independent signal** alongside the deterministic polygon geo-fencing.
- Research / prototyping on GeoLife-format trajectories.

Out of scope (do NOT do this)

- **Do not** treat the score as a probability of danger, crime, or risk.
- **Do not** use it as the sole trigger for real-world alerts.
- **Do not** apply it across populations it was not trained on without re-evaluation — GeoLife is dominated by a few Beijing users, so "normal" is heavily biased to that region and those people.
- **Do not** use it for surveillance or to infer sensitive attributes of individuals.

---

3. Features (14, order fixed)

```
speed_mps, log_dt_s, step_m, accel_mps2, turn_deg,
speed_mean_w5, speed_std_w5, step_mean_w5, turn_mean_w5, straightness_w5,
altitude, alt_change, tod_sin, tod_cos
```

- Per-step `dt/step/speed/bearing` come from `common.clean_points` (haversine distance, grouped by `traj_uid`).
- `*_w5` are 5-point rolling means/stds; `straightness_w5` is net-displacement / path-length over the window.
- `tod_sin` / `tod_cos` encode time-of-day cyclically.
- All features are standardised with the **training** scaler before scoring.
- NaN at segment starts is imputed to **0** in scaled space (i.e. the training mean) by `feature_matrix` .

---

4. Training data

Field	Value
Source	Microsoft GeoLife GPS Trajectories 1.3 (real)
Point sample	**1,429,740** cleaned points from **1,200** trajectories (fixed seed)
Split	**grouped by** `traj_uid` — 900 train / 300 test trajectories
Train points	**1,060,493**
Test points	**369,247**
Leakage check	train/test trajectory groups are **disjoint** (0 overlap); enforced in `train.py` and re-checked in `evaluate.py`

Grouping is by the composite key `user_id|trajectory_id` because 847 GeoLife file names collide across users.

Observed score distribution (real)

Split	mean	std	p50	p95	p99
train	0.4141	0.0479	0.4000	0.5056	0.5939
test	0.4080	0.0437	0.3959	0.4853	0.5810

The test distribution closely tracks train (no obvious drift on the held-out trajectories).

---

5. Evaluation — what can and cannot be measured

Supervised metrics are impossible. GeoLife has **no anomaly / danger ground truth**; its only labels are transportation modes. Accuracy, F1, precision, recall and a confusion matrix against *real* anomalies therefore **cannot be computed**. `evaluate.py` says so explicitly and reports none as if they existed.

What *is* computed (`ml/evaluate.py`, on the held-out, trajectory-disjoint test split), with **real data but synthetic labels**:

Metric	Value	Meaning
Protocol	GPS "teleport" burst injection	3 points moved ~2.2 km N / ~1.7 km E at known indices

Metric	Value	Meaning
Injected points affected	**17,462**	points marked anomalous
ROC-AUC	**0.8922**	detector separates injected teleports from normal movement
Average precision	**0.6763**	precision/recall trade-off for the rare positive class
Precision@k (k = 17,462)	**0.6165**	
Recall@k	**0.6165**	
Label-shuffle control ROC-AUC	**0.5007**	≈ 0.5, confirms the metric is meaningful, not an artefact

Baseline flagging thresholds (real)

Score quantile	Threshold	Fraction flagged
0.90	0.4588	10.00 %
0.95	0.4853	5.00 %
0.98	0.5299	2.00 %
0.99	0.5810	1.00 %
0.995	0.6374	0.50 %

Interpretation limit. A high injection ROC-AUC means the detector discriminates *teleport artefacts* well. It does **not** mean it detects real danger. Injection is a single anomaly family; other anomaly types (detours, unusual stops, wrong-way movement) are **not** covered by this experiment.

---

6. Explanation / interpretability

`ml/predict.py` explains the top anomalous points with two model-agnostic views (no SHAP dependency):

- **z-score** — how many training-std the feature lies from the training mean.
- **Leave-one-feature-out contribution** — the drop in anomaly score when that one feature is reset to the training mean (larger = stronger driver).

Example (real trajectory `20081023025304`, top point): `step_m` value 1817.5 m (z = +14.7) and `speed_std_w5` (z = +10.5) were the strongest drivers.

7. Limitations & known biases

1. **No ground truth** — quality against *real* anomalies is unknown and currently unmeasurable with GeoLife.
2. **Geographic/user bias** — data is dominated by a small set of Beijing users; "normal" is region- and person-specific.
3. **Anomaly ≠ danger** — unusual movement is frequently benign (GPS glitches, new routes, transport-mode changes).
4. **Altitude unreliable** — GeoLife altitude is noisy; features `altitude` and `alt_change` should be treated cautiously.
5. **Teleport injection is easy** — the 0.89 ROC-AUC is flattered by a very detectable anomaly; it is an upper-ish bound for that one family.
6. **Sampling** — training uses a 1,200-trajectory fixed-seed sample of the 18,670 available, to keep runtime/memory reasonable on a 7.9 GB machine.

---

8. Reproducibility / how to run

```
# from the repo root
pip install -r ml\requirements.txt
python ml\preprocess.py --summary-scope all --sample-trajectories 1200
python ml\eda.py
python ml\train.py
python ml\evaluate.py
python ml\predict.py --plt "ml\data\geolife\Geolife Trajectories 1.3\Data\000\Trajectory\2
python -m pytest ml\tests -q

# or the single driver
powershell -NoProfile -ExecutionPolicy Bypass -File ml\run_all.ps1
```

- Fixed seed **42**; pinned dependencies in `ml/requirements.txt`.
- If the real dataset is absent the pipeline uses the SYNTHETIC fixture and every stage prints a SYNTHETIC banner; **synthetic numbers are plumbing tests, not results**.